

```
bus = dbus.SessionBus()
obj = bus.get_object('im.pidgin.purple.PurpleService', '/im/pidgin/purple/PurpleObject')
purple = dbus.Interface(obj, 'im.pidgin.purple.PurpleInterface')
while True:
    fortune = subprocess.Popen('fortune', stdout=subprocess.PIPE).
stdout.read()
    set_status(fortune)
    time.sleep(10)
```

The above script makes use of the *fortune* command to generate random quotes. You may have noticed the gnome-panel applet Fish. Do you remember the "free the fish" Easter egg? Fish uses *fortune* as its back-end for generating quotes. The above script sets the status message for Pidgin every 10 seconds with a random quote generated by the *fortune* command.

The next application in line is Tomboy, a note-taking application, which ships with GNOME. This is how you can talk to Tomboy and collect all the notes created with it to print them on a terminal:

```
#!/usr/bin/env python

import dbus

bus = dbus.SessionBus()

obj = bus.get_object('org.gnome.Tomboy', '/org/gnome/Tomboy/RemoteControl')
tomboy = dbus.Interface(obj, 'org.gnome.Tomboy.RemoteControl')

notes = tomboy.ListAllNotes();

for note in notes:
    print tomboy.GetNoteContents(note)
```

How about fiddling with Exaile music player, the GNOME-based Amarok clone? Our aim is to write few lines of Bash script to enquire the application on current music track, album name and artist. Add the following lines to the *~/bashrc* file:

```
artist=$(dbus-send --print-reply --dest=org.exaile.dbusInterface \
/dbusInterfaceObject org.exaile.dbusInterface.get_artist 2> /dev/null | grep ".*" \
-o | tr -d '"');

album=$(dbus-send --print-reply --dest=org.exaile.dbusInterface \
/dbusInterfaceObject org.exaile.dbusInterface.get_album 2> /dev/null | grep \
".*" -o | tr -d '"');

if [[ -n $album ]]; then
echo -e "\nCurrently Playing $album, $artist\n";
fi
```

Notice how every time you open a new terminal, it lists the information about the song currently playing in Exaile. Of course, if the player is not running, it won't print anything. Here, the *dbus-send* command is used to communicate with Exaile through the D-Bus interface.

Finally, let's hack GNOME's PowerManager to hibernate our machine:

```
[slynnux@slynnux-laptop ~]$ python
Python 2.5.2 (r2526091.1, Sep 30 2008, 15:41:38)
[GCC 4.3.2 20080917 (Red Hat 4.3.2-4)] on linux2
Type "help", "copyright", "credits" or "license()" for more information.
>>>
>>> import dbus
>>> bus=dbus.SessionBus()
>>> power = bus.get_object('org.freedesktop.PowerManagement', '/org/freedesktop/PowerManagement')
>>> pm = dbus.Interface(power, 'org.freedesktop.PowerManagement')
>>> pm.Hibernate()
```

The above code makes the PowerManagement daemon execute the *Hibernate()* function and the machine goes into hibernation. You can also use the *Shutdown()*, *Reboot()*, *Suspend()* instead.

So you see, by embedding any kind of D-Bus interfacing you are able to extract different sorts of things from an application. There are numerous applications that are hackable with D-Bus interfacing. Try it out for yourself—it's fun!



Note: Debugging D-Bus applications can be a hurdle sometimes. You can use the *dbus-monitor* to examine the events for a better understanding. Alternatively, you can also check out the D-Feet D-Bus debugger tool written by John Palmeri.

Bottom line

Now a days, most of the GNOME and KDE apps come with dbus interface support. This makes it easier for applications to communicate with each other and eliminates the higher-degree task of recompiling every application to make it compatible with another.

Now, here is your task. You may find that some of your favourite applications do not have D-Bus support. If you do, maybe you can start writing the D-Bus interfacing for your favourite applications—contribute back to the community, it's not that hard really! Happy Hacking!



By: Sarath Lakshman

The author is a Hacktivist of Free and Open Source Software from Kerala. He loves working on the GNU/Linux environment and contributes to the PiTiVi video editor project. He is also the developer of SLYNIX, a distro for newbies. He blogs at www.sarathlakshman.info